Programa:	Cartoonist 1.1
Protección:	ASProtect 1.1c + nags + inserta watermark en las magenes creadas
Objetivos:	Eliminar las restricciones
URL descarga:	http://www.vicman.net/cartoonist/
Dificultad:	La IAT casi agota la paciencia del cracker ;-)
Herramientas:	OllyDbg v1.10 + PEiD 0.94 + ImpRec 1.6
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 04-01-06

Hola a todo CracksLatinoS!!

Lo prometido es deuda se suele decir así que aquí os mando la segunda parte del tute del cartoonist, en la primera se vio como desempakar manualmente al ASProtect 1.1c y quedaron pendientes las limitaciones y/o restricciones del programa, entre ellas:

1.- Nag inicial.
2.- Innserta watermark en las imágenes (marca "Unregistered version" muy fea)
3.- Nag final.

Estas son las dos más importantes, ni que decir tiene que lo que más molesta es el texto "Unregistered version" que pondrá en todos nuestros trabajos, una cosa que me sorprende es que no tiene limitación temporal cuando todos sabemos que los ASProtect suelen tomarse la licencia de ponerla, en este caso no lo hizo ¿?

Dicho esto podemos comenzar con el objetivo de hoy, quiero dedicar el tute a toda la lista de CracksLatinoS de la que cada día aprendo más, aunque suene a tópico es cierto, hay muchos buenos tutes allí en teorías y colaboraciones que todos deberíamos leer alguna vez. Y ahora sí pasemos al trabajo ;-)

Una cosa antes de continuar, debo decir que para seguir el tute será necesarios haber desempakado primero el ejecutable, en caso contrario no podríamos aplicar el parche en caso de que haya que hacerlo. Vamos a ello ;-)

Fulminando las restricciones...

Ejecutamos nuestro dumpeado y nos aparece esto ;-(

Vaya una nag! jeje.. esto a nosotros no nos gusta nada así que vamos por ella, cerramos la aplicación y nos abrimos un Olly, y alli lo cargamos, quedamos detenidos en el OEP:

```
Address   Hex dump    Disassembly                      Comment
00401000  $v EB 10    JMP SHORT tute_.00401012
00401002  .  66:623A  BOUND DI,DWORD PTR DS:[EDX]
00401005  .  43       INC EBX
00401006  .  2B2B     SUB EBP,DWORD PTR DS:[EBX]
00401008  .  48       DEC EAX
00401009  .  4F       DEC EDI
0040100A  .  4F       DEC EDI
0040100B  .  4B       DEC EBX
0040100C  .  90       NOP
0040100D  .- E9 68665D00  JMP 009D767A
00401012  > A1 5B665D00   MOV EAX,DWORD PTR DS:[5D665B]
```

recordamos de la primera parte cual era su valor 401000, desde aquí dejamos correr libremente al programa con F9 hasta que aprezca la nag que vimos en pantalla, es

entonces cuando pausamos el programa, bueno pausamos Olly en realidad, esto lo conseguimos con F12, hecho esto miramos en el stack (la pila), vemos esto:

```
0012F9F8  77D13C6B  RETURN to USER32.77D13C6B
0012F9FC  00557920  RETURN to tute_.00557920 from <JMP.&user32.WaitMessage>
0012FA00  0012FA54  Pointer to next SEH record
0012FA04  0055793B  SE handler
0012FA08  0012FA28
0012FA0C  0060FAE8  tute_.0060FAE8
0012FA10  00DB0E2C
0012FA14  00D91BA8
0012FA18  00000000
0012FA1C  00000000
0012FA20  0160FAE8
0012FA24  00D91BA8
0012FA28  0012FAA0
0012FA2C  005570D4  RETURN to tute_.005570D4 from tute_.0055784C
0012FA30  00110128
0012FA34  0000B001
0012FA38  00000000
0012FA3C  00000000
0012FA40  002D5805
0012FA44  0000016A
0012FA48  000000DF
0012FA4C  006288FC  tute_.006288FC
0012FA50  00553D2C  RETURN to tute_.00553D2C from tute_.005570B8
0012FA54  0012FA60  Pointer to next SEH record
0012FA58  00553DAF  SE handler
0012FA5C  0012FAA0
```

jeje.. bonita llamada!

ojo con la llamada resaltada, veamos como dice "from tute_.005570B8" esto quiere decir que desde allí se hizo la llamada, o sea que allí hay un CALL, si nos situamos sobre ella y pulsamos enter veremos esto en la ventana de código:

```
Address   Hex dump   Disassembly                          Comment
00553D2C  .  8B03    MOV EAX,DWORD PTR DS:[EBX]           ; aqui caemos..
00553D2E  .  80B8 8C00000(  CMP BYTE PTR DS:[EAX+8C],0
00553D35  .v 74 0F   JE SHORT tute_.00553D46
00553D37  .  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D3A  .  C780 3402000(  MOV DWORD PTR DS:[EAX+234],2
00553D44  .v EB 14   JMP SHORT tute_.00553D5A
00553D46  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D49  .  83B8 3402000(  CMP DWORD PTR DS:[EAX+234],0
00553D50  .v 74 08   JE SHORT tute_.00553D5A
00553D52  .  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D55  .  E8 26FDFFFF  CALL tute_.00553A80
00553D5A  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D5D  .  8B80 3402000(  MOV EAX,DWORD PTR DS:[EAX+234]
00553D63  .  85C0    TEST EAX,EAX
00553D65  .^ 74 BE   JE SHORT tute_.00553D25
00553D67  .  8945 F8 MOV DWORD PTR SS:[EBP-8],EAX
```

Esto es un bucle, si nos fijamos bien veremos que estamos en el típico buclecillo donde quedan las ventanas de diálogo "flotando" en pantalla, la nag no es ninguna excepción, lo que se hace es llamar a este loop que deja la ventana en pantalla a la espera de que hagamos algo con ella, si subimos un pelín con la rueda del ratón encontramos su inicio

```
Address   Hex dump   Disassembly                          Comment
00553D1D  .  33D2    XOR EDX,EDX
00553D1F  .  8990 3402000(  MOV DWORD PTR DS:[EAX+234],EDX
00553D25  > 8B03     MOV EAX,DWORD PTR DS:[EBX]           ; inicio loop..
00553D27  .  E8 8C330000  CALL tute_.005570B8
00553D2C  .  8B03    MOV EAX,DWORD PTR DS:[EBX]           ; aqui caemos..
00553D2E  .  80B8 8C00000(  CMP BYTE PTR DS:[EAX+8C],0
00553D35  .v 74 0F   JE SHORT tute_.00553D46
00553D37  .  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D3A  .  C780 3402000(  MOV DWORD PTR DS:[EAX+234],2
00553D44  .v EB 14   JMP SHORT tute_.00553D5A
00553D46  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D49  .  83B8 3402000(  CMP DWORD PTR DS:[EAX+234],0
00553D50  .v 74 08   JE SHORT tute_.00553D5A
00553D52  .  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D55  .  E8 26FDFFFF  CALL tute_.00553A80
00553D5A  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D5D  .  8B80 3402000(  MOV EAX,DWORD PTR DS:[EAX+234]
00553D63  .  85C0    TEST EAX,EAX
00553D65  .^ 74 BE   JE SHORT tute_.00553D25              ; fin loop..
00553D67  .  8945 F8 MOV DWORD PTR SS:[EBP-8],EAX
00553D6A  .  6A 00   PUSH 0
```

Ahora lo vemos completo, su inicio está en :00533D25, lo que haremos será poner un BP al final del mismo como se ve en la imagen y dejar correr nuevamente el programa con F9, la nag seguirá estando en pantalla, la cerramos pulsando "Try now" y Olly para:

```
Address   Hex dump   Disassembly                          Comment
00553D55  .  E8 26FDFFFF  CALL tute_.00553A80
00553D5A  >  8B45 FC MOV EAX,DWORD PTR SS:[EBP-4]
00553D5D  .  8B80 3402000(  MOV EAX,DWORD PTR DS:[EAX+234]
00553D63  .  85C0    TEST EAX,EAX
00553D65  .^ 74 BE   JE SHORT tute_.00553D25              ; fin loop..
00553D67  .  8945 F8 MOV DWORD PTR SS:[EBP-8],EAX
00553D6A  .  6A 00   PUSH 0
```

quedamos detenidos en el BP que hemos puesto, lo quitamos para que no vuelva a saltar cuando el progama nos saque algún cuadro de diálogo más, y comenzamos la búsqueda ;-) el método es bien sencillo se trata en ir traceando con F8 o bien ir pulsando Ctrl+F9 que lo que hace es ejcutar código hasta encontrar un RETN, de esta manera iremos más

rápido, lo que queremos es localizar la llamada a la nagscreen, es fácil de detectar pues siempre va precedido de una comparación (que suele ser el registro EAX):

```
Address   Hex dump   Disassembly                          Comment
004049CA  .  8B0F    MOV ECX,DWORD PTR DS:[EDI]
004049CC  .  8B01    MOV EAX,DWORD PTR DS:[ECX]
004049CE  .  8B10    MOV EDX,DWORD PTR DS:[EAX]
004049D0  .  FF92 D800000(  CALL DWORD PTR DS:[EDX+D8]   ; llama a la nag!
004049D6  .  83F8 02 CMP EAX,2                            ; que boton pulsamos?
004049D9  .v 75 0D   JNZ SHORT tute_.004049E8             ; salta segun lo que hicimos..
004049DB  .  8B0D 2CFB600(  MOV ECX,DWORD PTR DS:[60FB2C]  tute_.006288FC
004049E1  .  8B01    MOV EAX,DWORD PTR DS:[ECX]
004049E3  .  E8 18291500  CALL tute_.00557300
004049E8  > 8B55 C8  MOV EDX,DWORD PTR SS:[EBP-38]
004049EB  .  64:8915 0000(  MOV DWORD PTR FS:[0],EDX
```

En este caso la llamada está en :004049D0, ese "CMP EAX,2" es delator, lo que está mirando es ver qué botón pulsamos y actuar en consecuencia, es lógico, no se comportará de la misma manera si pulsamos por ejemplo "Try now" que pulsando "Exit" esto está claro, pero ¿cómo se entera el programa del botón que hemos pulsado? pues con este tipo de comprobaciones, normalmente EAX le retorna un valor en función del botón apretado, en este caso fue "Try now" y el retorno fue EAX==1, veámoslo:

```
Registers (FPU)
EAX 00000001
ECX 0012FAA0
EDX 00000000
EBX 0012FB5C
ESP 0012FAA8
EBP 0012FB04
```

pulsamos "Try now"..

Cuando pulsamos "Try.." el retorno es EAX==1, vale prefecto, pero sigamos. Si superamos el siguiente RETN llegamos a una zona donde parece que podemos evitar la nag:

```
Address   Hex dump   Disassembly                          Comment
004BCECD  .  8B83 6804000(  MOV EAX,DWORD PTR DS:[EBX+468]
004BCED3  .  E8 E07E0A00  CALL tute_.00564DB8
004BCED8  .  80BB F002000(  CMP BYTE PTR DS:[EBX+2F0],0   ; hmmm...
004BCEDF  .v 75 72   JNZ SHORT tute_.004BCF53
004BCEE1  .  66:C746 10 0(  MOV WORD PTR DS:[ESI+10],8
004BCEE7  .  8D4D F0 LEA ECX,DWORD PTR SS:[EBP-10]
004BCEEA  .  51      PUSH ECX                             Arg1
004BCEEB  .  E8 AC76F4FF  CALL tute_.0040459C             tute_.0040459C
004BCEF0  .  59      POP ECX
004BCEF1  .  8D45 F0 LEA EAX,DWORD PTR SS:[EBP-10]
004BCEF4  .  8346 1C 05  ADD DWORD PTR DS:[ESI+1C],5
004BCEF8  .  66:C746 10 1(  MOV WORD PTR DS:[ESI+10],14
004BCEFE  .  50      PUSH EAX                             Arg1
004BCEFF  .  E8 E878F4FF  CALL tute_.004047EC             tute_.004047EC
004BCF04  .  59      POP ECX                              0012FB5C
004BCF05  .  FF4E 1C DEC DWORD PTR DS:[ESI+1C]
```

El salto que hay en :004BDEDF parece ser bueno pues de producirse nos alejaríamos de la dirección :004BDEFF donde nos muestra la nagscreen, por si acaso pondremos otro bp allí:

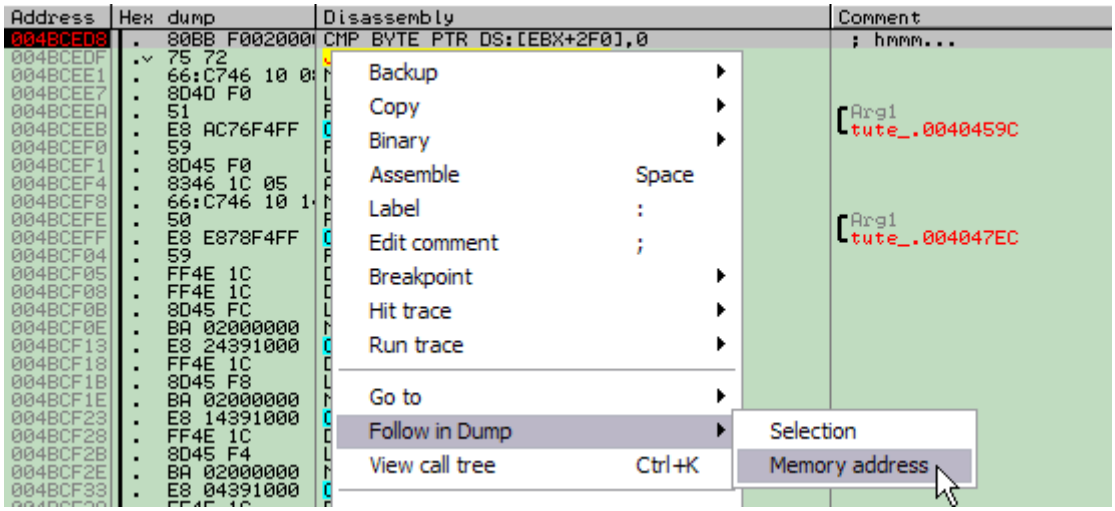
```
Address   Hex dump   Disassembly                          Comment
004BCECD  .  8B83 6804000(  MOV EAX,DWORD PTR DS:[EBX+468]
004BCED3  .  E8 E07E0A00  CALL tute_.00564DB8
004BCED8  .  80BB F002000(  CMP BYTE PTR DS:[EBX+2F0],0   ; hmmm...
004BCEDF  .v 75 72   JNZ SHORT tute_.004BCF53
004BCEE1  .  66:C746 10 0(  MOV WORD PTR DS:[ESI+10],8
```

y continuamos traceando, al llegar al salto que hay poco después (en :004BCF4E) vemos referencias claras a "LicensedTo" lo que confirma nuestras sospechas de que el salto de :004BDEDF es bastante bueno. Como tenemos un BP en él reiniciaremos Olly con Ctr+F2 y dejaremos correr el programa, al poco estamos allí y lo primero que haremos será ver a qué posición de memoria hace referencia la variable [EBX+2F0], esto lo hacemos posicionándonos sobre ella y picando con el botón derecho sobre ella, en el menú flotante que aparece elejir "Follow in dump->Memory address" veremos esto:

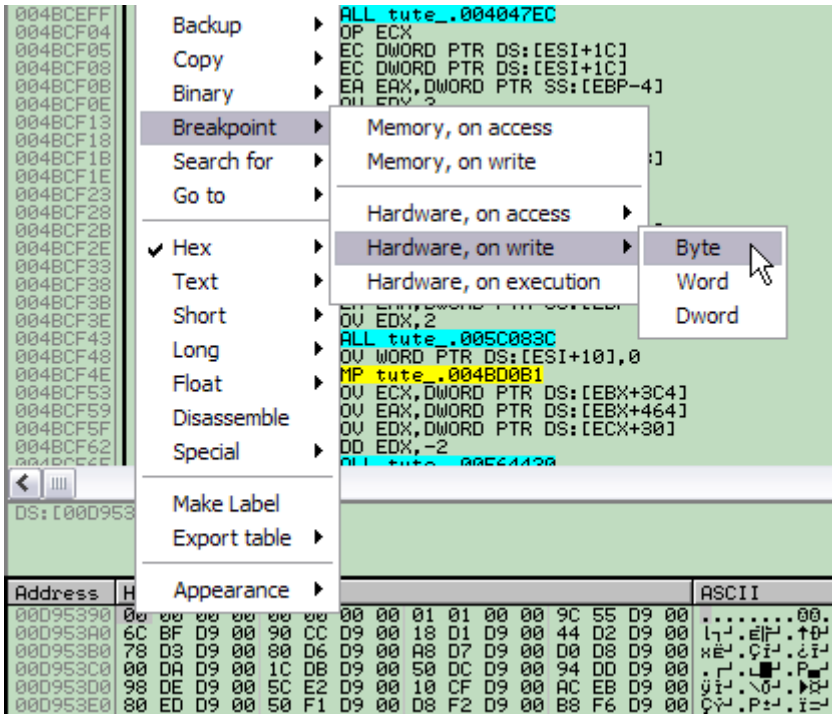
```
Address   Hex dump                                         ASCII
00D95390  00 00 00 00 00 00 00 00 01 01 00 00 9C 55 D9 00  ........00..£U┘.
00D953A0  6C BF D9 00 90 CC D9 00 18 D1 D9 00 44 D2 D9 00  l┐┘.É╠┘.↑╤┘.DÒ┘.
00D953B0  78 D3 D9 00 80 D6 D9 00 A8 D7 D9 00 D0 D8 D9 00  xÓ┘.Ç╓┘.¿╫┘.╨╪┘.
00D953C0  00 DA D9 00 1C DB D9 00 50 DC D9 00 94 DD D9 00  .┌┘.└█┘.P▄┘.ö¦┘.
```

Hace referencia a [D95390] una posición de memoria alejada del ejecutable, forma parte de un bloque de memoria creada en tiempo de ejecución, pero esto no quiere decir nada porque podemos parchearlo si nos hiciera falta, lo que vamos a hacer va más allá de cambiar un solo salto, vamos a tratar de localizar el momento en el que el flag se inicializa, después seguiremos viendo las operaciones que hace y el valor que va almacenando en el mismo a partir de los resultados obtenidos, de esta manera conseguiremos casi en un 90% localizar la rutina de protección del programa.

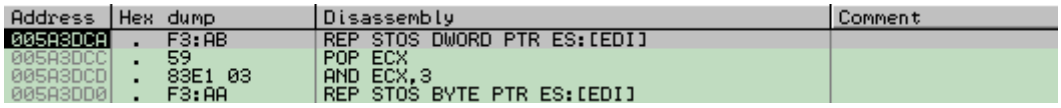
Esto lo haremos de la siguiente manera: Nos situamos sobre la línea del flag perseguido [EBX+468] y hacemos un "Follow in dump->Memory Address"



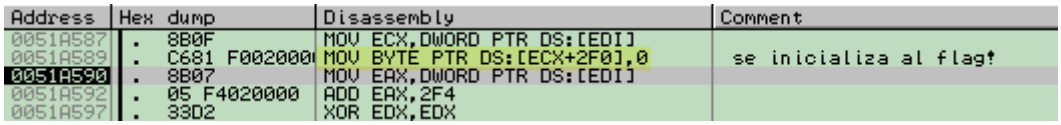
En la ventana de dumpp tendremos los bytes a la vista, seleccionamos el byte bajo y le ponemos un "HBP On Write->Dword" como en la imagen:



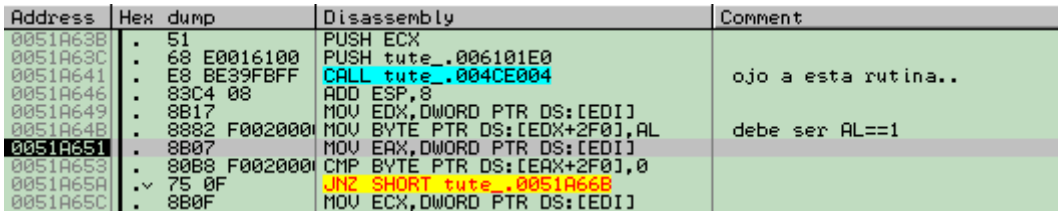
Y el HBP queda bien puesto y en su sitio ;-) reiniciamos la aplicación como siempre con Ctrl+F2 y paramos en el OEP del programa, pulsamos F9 para dejarlo correr y Olly para:



Pero no vemos nada interesante, F9 de nuevo y vuelve a parar:



En este momento se está inicializando la variable, la prepara de alguna manera para que esté a 0 desde un primer momento, después valorará si le pone o no el "deseado" 1, jeje.. pulsamos F9 otra vez y de nuevo para:



Esta vez atinamos en el blanco, una llamada de la que depende directamente el valor de AL y acto seguido la modificación de nuestro flag, justo lo que buscábamos, quitamos el HBP y ponemos un BP en el CALL:

```
Address   Hex dump   Disassembly                  Comment
0051A63B  .  51      PUSH ECX
0051A63C  .  68 E0016100  PUSH tute_.006101E0
0051A641  .  E8 BE39FBFF  CALL tute_.004CE004     ojo a esta rutina..
0051A646  .  83C4 08     ADD ESP,8
0051A649  .  8B17        MOV EDX,DWORD PTR DS:[EDI]
```

Reiniciamos Olly y dejamos correr el programa con F9, al poco parará aquí, entramos dentro con F7 y miramos la zona mientras la traceamos, no es muy grande y al final de la misma en lugar de haber un RETN hay un JMP que nos pone en una continuación de la misma, seguiremos este JMP a ver donde nos lleva:

```
Address   Hex dump   Disassembly                  Comment
004CE03C  .  8D45 FC     LEA EAX,DWORD PTR SS:[EBP-4]
004CE03F  .  BA 02000000 MOV EDX,2
004CE044  .  E8 F3270F00 CALL tute_.005C083C
004CE049  .v E9 4A000000 JMP tute_.004CE098       seguimos este JMP..
004CE04E     09          DB 09
```

Y nos pone en una zona donde sí vemos un RETN, jeje.. este será el final de la rutina, por lo tanto en este pequeño tramo se decide nuestra "suerte" si somos atentos veremos además que intentan despistarnos, parece ser que EAX toma su valor del CALL anterior pero no es así, miremos bien por encima porque poco antes de éste (del call) hay un "PUSH EAX" que es el que realmente decide qué valor nos llevaremos de la rutina, recuperándolo poco después en el "POP EAX" que es la instrucción contraria:

```
Address   Hex dump   Disassembly                  Comment
004CE098  > 66:C745 E0 2( MOV WORD PTR SS:[EBP-20],20   el JMP nos pone aqui..
004CE09E  .  BA 7ECE5D00 MOV EDX,tute_.005DCE7E
004CE0A3  .  8D45 F4     LEA EAX,DWORD PTR SS:[EBP-C]
004CE0A6  .  E8 41260F00 CALL tute_.005C06EC
004CE0AB  .  FF45 EC     INC DWORD PTR SS:[EBP-14]
004CE0AE  .  8D55 F4     LEA EDX,DWORD PTR SS:[EBP-C]
004CE0B1  .  8BC6        MOV EAX,ESI
004CE0B3  .  E8 6C280F00 CALL tute_.005C0924
004CE0B8  .  50          PUSH EAX                   ojo a este PUSH..
004CE0B9  .  FF4D EC     DEC DWORD PTR SS:[EBP-14]
004CE0BC  .  8D45 F4     LEA EAX,DWORD PTR SS:[EBP-C]
004CE0BF  .  BA 02000000 MOV EDX,2
004CE0C4  .  E8 73270F00 CALL tute_.005C083C
004CE0C9  .  58          POP EAX                    aqui el POP..
004CE0CA  .  8B55 D0     MOV EDX,DWORD PTR SS:[EBP-30]
004CE0CD  .  64:67:8916 0( MOV DWORD PTR FS:[0],EDX
004CE0D3  .  5E          POP ESI
004CE0D4  .  5B          POP EBX
004CE0D5  .  8BE5        MOV ESP,EBP
004CE0D7  .  5D          POP EBP
004CE0D8  .  C3          RETN                       Y retornamos..
004CE0D9     90          NOP
```

De esto concluimos que el call que debe retornar AL==1 está en :4CE0B3 o sea que allí tenemos la rutina de protección del programa, en ella se verifican las condiciones de si somos o no usuarios registrados, fijáos que la hemos encontrado aún sin introducir ningún serial, cosa que no es estrictamente necesaria.

El siguiente punto será forzar a dicho call a retornar AL==1 como único valor siempre que esta llamada sea invocada desde el programa principal, normalmente hace una llamada para cada nag, restricción, limitación, etc., por esto mismo si modificamos el valor de retorno al que nos vaya bien tendremos el programa totalmente desprotegido. Entramos dentro con F7 y echamos un vistazo:

```
Address   Hex dump   Disassembly                  Comment
005C0924  $  55      PUSH EBP
005C0925  .  8BEC    MOV EBP,ESP
005C0927  .  53      PUSH EBX
005C0928  .  8B00    MOV EAX,DWORD PTR DS:[EAX]     primera cadena a comparar..
005C092A  .  8B12    MOV EDX,DWORD PTR DS:[EDX]     segunda cadena a comparar..
005C092C  .  E8 F342FEFF CALL tute_.005A4C24        compara cadenas..
005C0931  .  0F95C0  SETNE AL                       debe quedar AL==1
005C0934  .  83E0 01 AND EAX,1
005C0937  .  5B      POP EBX
005C0938  .  5D      POP EBP
005C0939  .  C3      RETN
```

Aquí lo que se hace es empujar a la pila una cadena (en :5C0928) y después empujar otra (en :5C092A), acto seguido se mete en un call y las compara a ver si son iguales (en :5C092C) lo más extraño es que traceé la rutina y las dos cadenas eran dos argumentos vacíos (NULL) quizá porque no le hemos introducido ningún serial, pero esto no importa, podríamos modificar el "SETNE AL" por un "SETE AL" y listos pero si por casualidad el resultado del call fuera el esperado estaríamos modificándolo, como esto no interesa lo que haremos será forzar directamente el valor de ALL para que siempre valga 1 como resultado de retorno, esto lo conseguimos muy fácil:

```
Address   Hex dump     Disassembly                  Comment
005C0924  $ 55         PUSH EBP
005C0925  . 8BEC       MOV EBP,ESP
005C0927  . 53         PUSH EBX
005C0928  . 8B00       MOV EAX,DWORD PTR DS:[EAX]   primera cadena a comparar..
005C092A  . 8B12       MOV EDX,DWORD PTR DS:[EDX]   segunda cadena a comparar..
005C092C  . E8 F342FEFF CALL tute_.005A4C24         compara cadenas..
005C0931  . 0F95C0     SETNE AL                     debe quedar AL==1
005C0934    33C0       XOR EAX,EAX
005C0936    40         INC EAX
005C0937  . 5B         POP EBX
005C0938  . 5D         POP EBP
005C0939  . C3         RETN
```

Lo único que hace nuestro parche es poner EAX a cero e incrementarlo en 1 ud. quedando lógicamente EAX==1 (por consiguiente AL==1) que es el resultado esperado, salvamos los cambios seleccionando los bytes cambiados y "Copy to executable->Selection" después "Save file" le damos otro nombre, "tute_crk.exe" por ejemplo.

Hasta aquí todo claro, sólo nos queda probar nuestra obra de arte ;-) hacemos doble clic sobre el nuevo .exe y funciona a las mil maravillas, las limitaciones han desaparecido todas, entramos y salimos sin ver las nags y nos salva las imagenes sin añadirle el "Unregistered version" de antes.

Esto quiere decir que nuestro crack funciona ;-)

Y hasta aquí la parte2, no dio para más, espero eso sí que os haya gustado el tute, y sobretodo a los más nuevos se les haya quedado el método de búsqueda de la rutina de protección de un programa a través de las nags recordatorio que nos envía el programa.

Un saludo a los CracksLatinoS y hasta la próxima, un placer

SyXe'05

NOTA: Para cualquier cosa ya sabéis donde encontrarme: syxe00@yahoo.es, os espero!